A6B2_EXPT-9: WHITE BOX TESTING

Basis Path Testing with JUnit - Java

Particular	Details
Course	Software Engineering Lab (24CS01PR0502)
Experiment	9 - White Box Testing
Name	Vineet Shah
Class / Batch	5S CSE / A6 - B2
Roll No.	28
Date of execution	25-10-2026
Case study	Online Leave Management System
Faculty	D A Borikar

Problem Statement [in Java]

[1] Implement a Class as mentioned during the lab instructions. Create a JUnit Test Suite incorporating all possible test cases covering independent paths.

[2] Choose any class from your Class Diagram (one which involves substantial computational logic) and implement it. Now create JUnit suite to test this class appropriately.

Aim

To implement Java classes, draw the control flow graph of each method under test, compute its cyclomatic complexity, identify the independent paths, and create a JUnit test suite with one test case for every independent path.

Theory

White box testing designs test cases from the internal structure of the code. In basis path testing the code is converted into a control flow graph (CFG): every node is a statement or a condition and every edge is a possible transfer of control. A compound condition such as (a <= 0 || b <= 0) is split into one node per simple condition, because Java evaluates it with short-circuiting.

- Cyclomatic complexity V(G) = E - N + 2P, where E = edges, N = nodes and P = connected components (P = 1 for one method).
- Equivalently, V(G) = number of predicate (decision) nodes + 1.
- V(G) is the number of linearly independent paths; each independent path adds at least one edge not covered by the earlier paths. Testing all of them executes every statement and every branch.
- JUnit 5: @Test marks a test method, assertEquals(expected, actual) checks the result, @DisplayName names the test, and @Suite with @SelectClasses groups test classes into a test suite.

Tools Used

Tool	Version / use
Java	Eclipse Temurin JDK 21.0.12 (javac / java)
JUnit	JUnit 5.11.4 (Jupiter) with JUnit Platform 1.11.4 Suite engine, run through the junit-platform-console-standalone launcher
JaCoCo	0.8.12 - records which lines and branches were executed by the test suite
Run script	run_tests.ps1 - compiles src/ and test/, runs WhiteBoxTestSuite, writes reports/

Part 1: Triangle Class

Triangle.classify(a, b, c) takes the three sides of a triangle and returns INVALID (a side is zero or negative), NOT A TRIANGLE (triangle inequality fails), EQUILATERAL, ISOSCELES or SCALENE.

1.1 Source code - Triangle.java

```
1  /**
2   * Part 1: classifies a triangle from the lengths of its three sides.
3   */
4  public class Triangle {
5
6      public static String classify(int a, int b, int c) {
7          if (a <= 0 || b <= 0 || c <= 0) {
8              return "INVALID";
9          }
10         if (a + b <= c || b + c <= a || a + c <= b) {
11             return "NOT A TRIANGLE";
12         }
13         if (a == b) {
14             if (b == c) {
15                 return "EQUILATERAL";
16             }
17             return "ISOSCELES";
18         }
19         if (b == c || a == c) {
20             return "ISOSCELES";
21         }
22         return "SCALENE";
23     }
24 }
```

1.2 Control Flow Graph

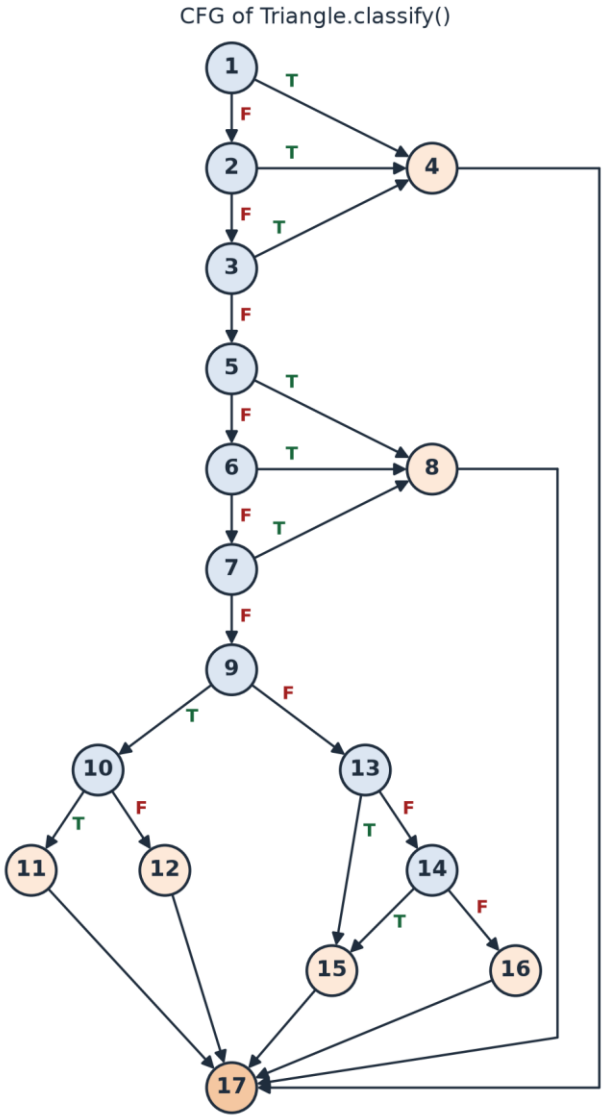


Figure 1: CFG of Triangle.classify() (blue = decision node, orange = return / exit node)

Node	Line	Statement / condition
1	7	a <= 0
2	7	b <= 0
3	7	c <= 0
4	8	return "INVALID"
5	10	a + b <= c
6	10	b + c <= a
7	10	a + c <= b
8	11	return "NOT A TRIANGLE"
9	13	a == b
10	14	b == c
11	15	return "EQUILATERAL"
12	17	return "ISOSCELES"
13	19	b == c
14	19	a == c
15	20	return "ISOSCELES"
16	22	return "SCALENE"
17	23	end of method (exit)

1.3 Cyclomatic Complexity

Method	Calculation	V(G)
E - N + 2P	E = 26 edges, N = 17 nodes, P = 1 -> 26 - 17 + 2	11
Predicate nodes + 1	Decision nodes 1, 2, 3, 5, 6, 7, 9, 10, 13, 14 = 10 -> 10 + 1	11
JaCoCo report	Complexity reported for classify(int, int, int)	11

Hence there are 11 independent paths and at least 11 test cases are required.

1.4 Independent Paths and Test Cases

Path	Node sequence	Input (a, b, c)	Expected output	Actual output	Result
P1	1-4-17	(0, 5, 5)	INVALID	INVALID	**PASS**
P2	1-2-4-17	(4, 0, 3)	INVALID	INVALID	**PASS**
P3	1-2-3-4-17	(4, 3, -2)	INVALID	INVALID	**PASS**
P4	1-2-3-5-8-17	(1, 2, 3)	NOT A TRIANGLE	NOT A TRIANGLE	**PASS**
P5	1-2-3-5-6-8-17	(7, 2, 3)	NOT A TRIANGLE	NOT A TRIANGLE	**PASS**
P6	1-2-3-5-6-7-8-17	(2, 8, 3)	NOT A TRIANGLE	NOT A TRIANGLE	**PASS**
P7	1-2-3-5-6-7-9-10-11-17	(5, 5, 5)	EQUILATERAL	EQUILATERAL	**PASS**
P8	1-2-3-5-6-7-9-10-12-17	(5, 5, 8)	ISOSCELES	ISOSCELES	**PASS**
P9	1-2-3-5-6-7-9-13-15-17	(4, 6, 6)	ISOSCELES	ISOSCELES	**PASS**
P10	1-2-3-5-6-7-9-13-14-15-17	(6, 4, 6)	ISOSCELES	ISOSCELES	**PASS**
P11	1-2-3-5-6-7-9-13-14-16-17	(3, 4, 5)	SCALENE	SCALENE	**PASS**

Actual output is the value returned by classify() during the JUnit run; assertEquals passed for every path (see the console output in Section 3).

1.5 JUnit test class - TriangleTest.java

```
1  import static org.junit.jupiter.api.Assertions.assertEquals;
2
3  import org.junit.jupiter.api.DisplayName;
4  import org.junit.jupiter.api.MethodOrderer;
5  import org.junit.jupiter.api.Order;
6  import org.junit.jupiter.api.Test;
7  import org.junit.jupiter.api.TestMethodOrder;
8
9  /**
10  * Basis-path tests for Triangle.classify(): one test per independent path
11  * of the control flow graph (V(G) = 11). Node numbers refer to the CFG.
12  */
13 @DisplayName("Part 1 - Triangle.classify() independent paths")
14 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
15 class TriangleTest {
16
17     @Test
18     @Order(1)
19     @DisplayName("P1: 1-4-17  side a <= 0")
20     void path1SideANotPositive() {
21         assertEquals("INVALID", Triangle.classify(0, 5, 5));
22     }
23
24     @Test
25     @Order(2)
26     @DisplayName("P2: 1-2-4-17  side b <= 0")
27     void path2SideBNotPositive() {
28         assertEquals("INVALID", Triangle.classify(4, 0, 3));
29     }
30
31     @Test
32     @Order(3)
33     @DisplayName("P3: 1-2-3-4-17  side c <= 0")
34     void path3SideCNotPositive() {
35         assertEquals("INVALID", Triangle.classify(4, 3, -2));
36     }
37
38     @Test
39     @Order(4)
40     @DisplayName("P4: 1-2-3-5-8-17  a + b <= c")
41     void path4FirstInequalityFails() {
```

```
42          assertEquals("NOT A TRIANGLE", Triangle.classify(1, 2, 3));
43      }
44
45      @Test
46      @Order(5)
47      @DisplayName("P5: 1-2-3-5-6-8-17  b + c <= a")
48      void path5SecondInequalityFails() {
49          assertEquals("NOT A TRIANGLE", Triangle.classify(7, 2, 3));
50      }
51
52      @Test
53      @Order(6)
54      @DisplayName("P6: 1-2-3-5-6-7-8-17  a + c <= b")
55      void path6ThirdInequalityFails() {
56          assertEquals("NOT A TRIANGLE", Triangle.classify(2, 8, 3));
57      }
58
59      @Test
60      @Order(7)
61      @DisplayName("P7: 1-2-3-5-6-7-9-10-11-17  a == b == c")
62      void path7Equilateral() {
63          assertEquals("EQUILATERAL", Triangle.classify(5, 5, 5));
64      }
65
66      @Test
67      @Order(8)
68      @DisplayName("P8: 1-2-3-5-6-7-9-10-12-17  a == b, b != c")
69      void path8IsoscelesAEqualsB() {
70          assertEquals("ISOSCELES", Triangle.classify(5, 5, 8));
71      }
72
73      @Test
74      @Order(9)
75      @DisplayName("P9: 1-2-3-5-6-7-9-13-15-17  b == c")
76      void path9IsoscelesBEqualsC() {
77          assertEquals("ISOSCELES", Triangle.classify(4, 6, 6));
78      }
79
80      @Test
81      @Order(10)
82      @DisplayName("P10: 1-2-3-5-6-7-9-13-14-15-17  a == c")
83      void path10IsoscelesAEqualsC() {
84          assertEquals("ISOSCELES", Triangle.classify(6, 4, 6));
85      }
86
87      @Test
88      @Order(11)
89      @DisplayName("P11: 1-2-3-5-6-7-9-13-14-16-17  all sides different")
90      void path11Scalene() {
91          assertEquals("SCALENE", Triangle.classify(3, 4, 5));
92      }
93  }
```

Part 2: LeaveService Class (from the Class Diagram)

From the class diagram of the Online Leave Management System, the LeaveService class is chosen because it holds the main computational logic of the system: it counts the leave days of a request (a loop over the dates that skips Sundays), applies the leave policy (contract faculty are not eligible for Medical Leave; per-request limits of CL 3, ML 15 and OD 10 days), checks the faculty member's balance, and updates the balance when the request is approved. It uses the Faculty class, which stores the category and the leave balance of a faculty member.

2.1 Source code - Faculty.java

```
import java.util.HashMap;
import java.util.Map;

/**
 * A faculty member of the Online Leave Management System with a category
 * (PERMANENT or CONTRACT) and an available balance for each leave type (CL, ML, OD).
 */
public class Faculty {

    private final String id;
    private final String category;
    private final Map<String, Integer> balance = new HashMap<>();

    public Faculty(String id, String category, int clBalance, int mlBalance, int odBalance) {
        this.id = id;
        this.category = category;
        balance.put("CL", clBalance);
        balance.put("ML", mlBalance);
        balance.put("OD", odBalance);
    }

    public String getId() {
        return id;
    }

    public boolean isContract() {
        return "CONTRACT".equals(category);
    }

    public int getBalance(String leaveType) {
        return balance.get(leaveType);
    }

    public void deduct(String leaveType, int days) {
        balance.put(leaveType, balance.get(leaveType) - days);
    }
}
```

2.2 Source code - LeaveService.java

```
import java.time.DayOfWeek;
import java.time.LocalDate;

/**
 * Part 2: Leave Service of the Online Leave Management System.
 * Counts the leave days of a request, checks it against the leave policy,
 * approves or rejects it, and updates the faculty member's leave balance.
 */
public class LeaveService {

    public static final int CL_LIMIT = 3;
    public static final int ML_LIMIT = 15;
    public static final int OD_LIMIT = 10;

    /** Number of leave days from 'from' to 'to' (both inclusive); Sundays are not counted. */
    public int countLeaveDays(LocalDate from, LocalDate to) {
        int days = 0;
        LocalDate day = from;
```

```
19          while (!day.isAfter(to)) {
20              if (day.getDayOfWeek() != DayOfWeek.SUNDAY) {
21                  days++;
22              }
23              day = day.plusDays(1);
24          }
25          return days;
26      }
27
28      /** Applies the leave policy to a request and returns the decision. */
29      public String processRequest(Faculty faculty, String leaveType, LocalDate from, LocalDate to) {
30          int limit;
31          if (leaveType.equals("CL")) {
32              limit = CL_LIMIT;
33          } else if (leaveType.equals("ML")) {
34              if (faculty.isContract()) {
35                  return "REJECTED: contract faculty are not eligible for ML";
36              }
37              limit = ML_LIMIT;
38          } else if (leaveType.equals("OD")) {
39              limit = OD_LIMIT;
40          } else {
41              return "REJECTED: unknown leave type " + leaveType;
42          }
43
44          int days = countLeaveDays(from, to);
45          if (days == 0) {
46              return "REJECTED: no leave days in the selected period";
47          }
48          if (days > limit) {
49              return "REJECTED: " + leaveType + " cannot exceed " + limit + " days per request";
50          }
51          if (days > faculty.getBalance(leaveType)) {
52              return "REJECTED: insufficient " + leaveType + " balance";
53          }
54          faculty.deduct(leaveType, days);
55          int left = faculty.getBalance(leaveType);
56          return "APPROVED: " + days + " day(s) of " + leaveType + ", balance left " + left;
57      }
58  }
```

2.3 Method countLeaveDays(from, to)

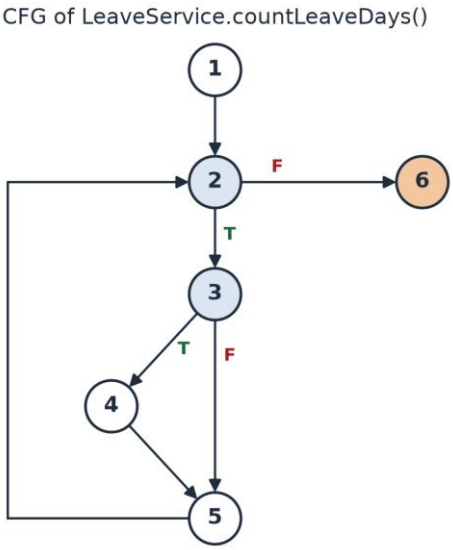


Figure 2: CFG of LeaveService.countLeaveDays()

Node	Line	Statement / condition
1	17-18	days = 0; day = from
2	19	while (!day.isAfter(to))
3	20	if (day.getDayOfWeek() != SUNDAY)
4	21	days++
5	23	day = day.plusDays(1)
6	25	return days (exit)

Method	Calculation	V(G)
E - N + 2P	E = 7, N = 6, P = 1 -> 7 - 6 + 2	3
Predicate nodes + 1	Decision nodes 2, 3 = 2 -> 2 + 1	3
JaCoCo report	Complexity reported for countLeaveDays(LocalDate, LocalDate)	3

Path	Node sequence	Input	Expected	Actual	Result
Q1	1-2-6	from = 02-11-2026 (Mon), to = 01-11-2026	0	0	**PASS**
Q2	1-2-3-4-5-2-6	from = to = 02-11-2026 (Mon)	1	1	**PASS**
Q3	1-2-3-5-2-6	from = to = 01-11-2026 (Sun)	0	0	**PASS**
Extra	1-(2-3-4/5)x7-2-6	from = 02-11-2026, to = 08-11-2026 (loop runs 7 times)	6	6	**PASS**

Q1-Q3 are the 3 independent paths. The extra test runs the loop many times (a full week) to check the loop body repeatedly; 01-11-2026 and 08-11-2026 are Sundays.

2.4 Method processRequest(faculty, leaveType, from, to)

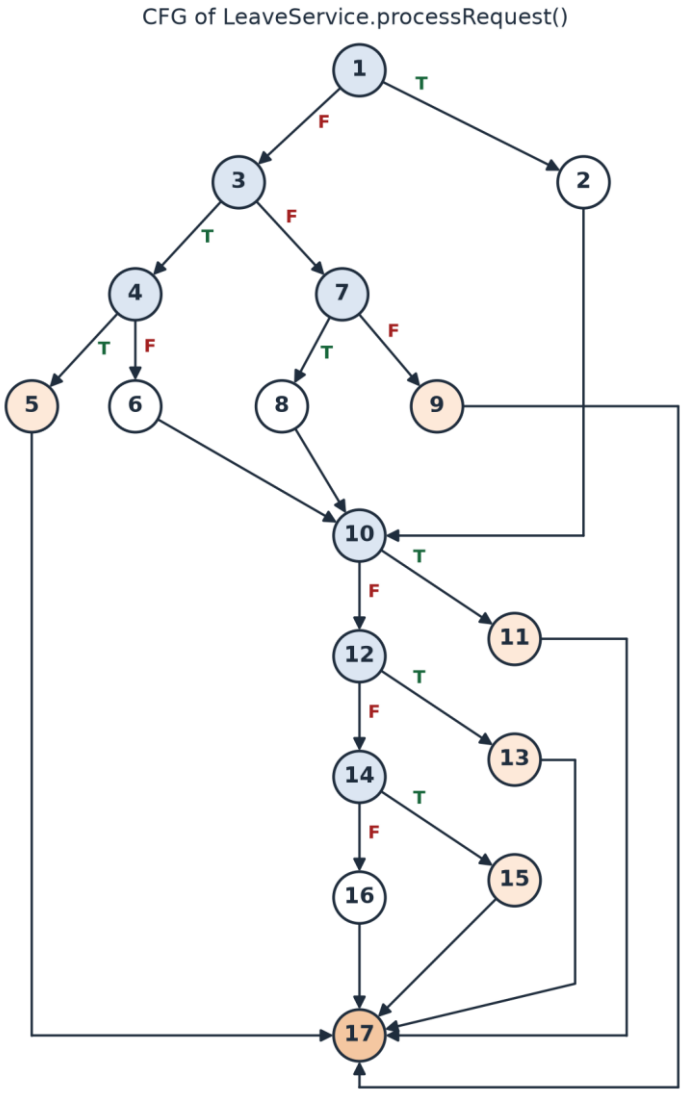


Figure 3: CFG of LeaveService.processRequest()

Node	Line	Statement / condition
1	31	leaveType.equals("CL")
2	32	limit = CL_LIMIT
3	33	leaveType.equals("ML")
4	34	faculty.isContract()
5	35	return "REJECTED: contract faculty ..."
6	37	limit = ML_LIMIT
7	38	leaveType.equals("OD")
8	39	limit = OD_LIMIT
9	41	return "REJECTED: unknown leave type ..."
10	44-45	days = countLeaveDays(from, to); days == 0
11	46	return "REJECTED: no leave days ..."
12	48	days > limit
13	49	return "REJECTED: ... cannot exceed ..."
14	51	days > faculty.getBalance(leaveType)
15	52	return "REJECTED: insufficient ... balance"
16	54-56	faculty.deduct(...); left = ...; return "APPROVED: ..."
17	57	end of method (exit)

Method	Calculation	V(G)
E - N + 2P	E = 23, N = 17, P = 1 -> 23 - 17 + 2	8
Predicate nodes + 1	Decision nodes 1, 3, 4, 7, 10, 12, 14 = 7 -> 7 + 1	8
JaCoCo report	Complexity reported for processRequest(Faculty, String, LocalDate, LocalDate)	8

Test data: FAC101 is PERMANENT with balance CL 8, ML 10, OD 7; FAC102 is CONTRACT; FAC103 is PERMANENT with CL balance 1. A new LeaveService and new Faculty objects are created before every test (@BeforeEach).

Path	Node sequence	Input	Expected output	Actual output	Result
P1	1-2-10-12-14-16-17	FAC101 (Permanent), CL, 02-11-2026 to 03-11-2026	APPROVED: 2 day(s) of CL, balance left 6	APPROVED: 2 day(s) of CL, balance left 6	PASS
P2	1-3-4-5-17	FAC102 (Contract), ML, 02-11-2026 to 02-11-2026	REJECTED: contract faculty are not eligible for ML	REJECTED: contract faculty are not eligible for ML	PASS
P3	1-3-4-6-10-12-14-16-17	FAC101 (Permanent), ML, 02-11-2026 to 06-11-2026	APPROVED: 5 day(s) of ML, balance left 5	APPROVED: 5 day(s) of ML, balance left 5	PASS
P4	1-3-7-8-10-12-14-16-17	FAC101 (Permanent), OD, 02-11-2026 to 04-11-2026	APPROVED: 3 day(s) of OD, balance left 4	APPROVED: 3 day(s) of OD, balance left 4	PASS
P5	1-3-7-9-17	FAC101, EL, 02-11-2026 to 02-11-2026	REJECTED: unknown leave type EL	REJECTED: unknown leave type EL	PASS
P6	1-2-10-11-17	FAC101, CL, 01-11-2026 (Sun) to 01-11-2026	REJECTED: no leave days in the selected period	REJECTED: no leave days in the selected period	PASS
P7	1-2-10-12-13-17	FAC101, CL, 02-11-2026 to 05-11-2026 (4 days)	REJECTED: CL cannot exceed 3 days per request	REJECTED: CL cannot exceed 3 days per request	PASS
P8	1-2-10-12-14-15-17	FAC103 (CL balance 1), CL, 02-11-2026 to 03-11-2026	REJECTED: insufficient CL balance	REJECTED: insufficient CL balance	PASS
Extra	1-2-10-12-14-16-17	FAC101, CL, 31-10-2026 (Sat) to 02-11-2026 (Mon)	APPROVED: 2 day(s) of CL, balance left 6	APPROVED: 2 day(s) of CL, balance left 6	PASS

P1-P8 are the 8 independent paths. The extra test checks the day calculation inside processRequest() when a Sunday falls inside the requested period.

2.5 JUnit test class - LeaveServiceTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;

import java.time.LocalDate;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.MethodOrderer;
import org.junit.jupiter.api.Nested;
import org.junit.jupiter.api.Order;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestMethodOrder;

/**
 * Basis-path tests for LeaveService. Node numbers refer to the control flow graphs:
 * countLeaveDays() has V(G) = 3 and processRequest() has V(G) = 8.
 * Dates used: 01-11-2026 is a Sunday, 02-11-2026 is a Monday.
 */
@DisplayName("Part 2 - LeaveService independent paths")
class LeaveServiceTest {

    private static final LocalDate SAT_31_OCT = LocalDate.of(2026, 10, 31);
    private static final LocalDate SUN_01_NOV = LocalDate.of(2026, 11, 1);
    private static final LocalDate MON_02_NOV = LocalDate.of(2026, 11, 2);

    private LeaveService service;
    private Faculty permanent;
    private Faculty contract;

    @BeforeEach
    void setUp() {
        service = new LeaveService();
        permanent = new Faculty("FAC101", "PERMANENT", 8, 10, 7);
        contract = new Faculty("FAC102", "CONTRACT", 5, 0, 5);
    }

```

```
    @Nested
    @DisplayName("countLeaveDays()  V(G) = 3")
    @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
    class CountLeaveDays {

        @Test
        @Order(1)
        @DisplayName("Q1: 1-2-6  loop not entered (To date before From date)")
        void pathQ1LoopNotEntered() {
            assertEquals(0, service.countLeaveDays(MON_02_NOV, SUN_01_NOV));
        }

        @Test
        @Order(2)
        @DisplayName("Q2: 1-2-3-4-5-2-6  one working day")
        void pathQ2OneWorkingDay() {
            assertEquals(1, service.countLeaveDays(MON_02_NOV, MON_02_NOV));
        }

        @Test
        @Order(3)
        @DisplayName("Q3: 1-2-3-5-2-6  one Sunday (not counted)")
        void pathQ3OneSunday() {
            assertEquals(0, service.countLeaveDays(SUN_01_NOV, SUN_01_NOV));
        }

        @Test
        @Order(4)
        @DisplayName("Loop many times: Mon 02-11 to Sun 08-11 = 6 days")
        void loopOverFullWeek() {
            assertEquals(6, service.countLeaveDays(MON_02_NOV, LocalDate.of(2026, 11, 8)));
        }
    }

    @Nested
    @DisplayName("processRequest()  V(G) = 8")
    @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
    class ProcessRequest {

        @Test
        @Order(1)
        @DisplayName("P1: 1-2-10-12-14-16-17  CL approved")
        void pathP1CasualLeaveApproved() {
            assertEquals("APPROVED: 2 day(s) of CL, balance left 6",
                    service.processRequest(permanent, "CL", MON_02_NOV, MON_02_NOV.plusDays(1)));
            assertEquals(6, permanent.getBalance("CL"));
        }

        @Test
        @Order(2)
        @DisplayName("P2: 1-3-4-5-17  ML requested by contract faculty")
        void pathP2ContractFacultyMedicalLeave() {
            assertEquals("REJECTED: contract faculty are not eligible for ML",
                    service.processRequest(contract, "ML", MON_02_NOV, MON_02_NOV));
        }

        @Test
        @Order(3)
        @DisplayName("P3: 1-3-4-6-10-12-14-16-17  ML approved for permanent faculty")
        void pathP3MedicalLeaveApproved() {
            assertEquals("APPROVED: 5 day(s) of ML, balance left 5",
                    service.processRequest(permanent, "ML", MON_02_NOV, MON_02_NOV.plusDays(4)));
        }

        @Test
        @Order(4)
        @DisplayName("P4: 1-3-7-8-10-12-14-16-17  OD approved")
        void pathP4OnDutyLeaveApproved() {
            assertEquals("APPROVED: 3 day(s) of OD, balance left 4",
                    service.processRequest(permanent, "OD", MON_02_NOV, MON_02_NOV.plusDays(2)));
        }

        @Test
        @Order(5)
        @DisplayName("P5: 1-3-7-9-17  unknown leave type")
        void pathP5UnknownLeaveType() {
            assertEquals("REJECTED: unknown leave type EL",
```

```
                        service.processRequest(permanent, "EL", MON_02_NOV, MON_02_NOV));
        }

        @Test
        @Order(6)
        @DisplayName("P6: 1-2-10-11-17  period has no leave days (only a Sunday)")
        void pathP6NoLeaveDays() {
            assertEquals("REJECTED: no leave days in the selected period",
                    service.processRequest(permanent, "CL", SUN_01_NOV, SUN_01_NOV));
        }

        @Test
        @Order(7)
        @DisplayName("P7: 1-2-10-12-13-17  CL of 4 days exceeds limit of 3")
        void pathP7LimitExceeded() {
            assertEquals("REJECTED: CL cannot exceed 3 days per request",
                    service.processRequest(permanent, "CL", MON_02_NOV, MON_02_NOV.plusDays(3)));
        }

        @Test
        @Order(8)
        @DisplayName("P8: 1-2-10-12-14-15-17  insufficient CL balance")
        void pathP8InsufficientBalance() {
            Faculty lowBalance = new Faculty("FAC103", "PERMANENT", 1, 10, 7);
            assertEquals("REJECTED: insufficient CL balance",
                    service.processRequest(lowBalance, "CL", MON_02_NOV, MON_02_NOV.plusDays(1)));
            assertEquals(1, lowBalance.getBalance("CL"));
        }

        @Test
        @Order(9)
        @DisplayName("Sat 31-10 to Mon 02-11: Sunday skipped, 2 days approved")
        void sundayInsideRangeIsNotCounted() {
            assertEquals("APPROVED: 2 day(s) of CL, balance left 6",
                    service.processRequest(permanent, "CL", SAT_31_OCT, MON_02_NOV));
        }
    }
}
```

3. JUnit Test Suite and Execution

3.1 Test suite - WhiteBoxTestSuite.java

```java
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;
import org.junit.platform.suite.api.SuiteDisplayName;

/**
 * JUnit test suite for Expt-9: runs the independent-path tests of both parts together.
 */
@Suite
@SuiteDisplayName("Expt-9 White Box Testing Suite")
@SelectClasses({ TriangleTest.class, LeaveServiceTest.class })
public class WhiteBoxTestSuite {
}
```

3.2 Command used

```
powershell -ExecutionPolicy Bypass -File run_tests.ps1

# inside run_tests.ps1 (test run):
java -javaagent:lib\jacoco-0.8.12\lib\jacocoagent.jar=destfile=out\jacoco.exec \
     -jar lib\junit-platform-console-standalone-1.11.4.jar execute \
     --class-path "out\classes;out\test-classes" --select-class WhiteBoxTestSuite \
     --details=tree
```

3.3 Console output (captured from the run on 25-10-2026)

```
╷
├─ JUnit Platform Suite ✓
│  └─ Expt-9 White Box Testing Suite ✓
│     └─ JUnit Jupiter ✓
│        ├─ Part 1 - Triangle.classify() independent paths ✓
│        │  ├─ P1: 1-4-17  side a <= 0 ✓
│        │  ├─ P2: 1-2-4-17  side b <= 0 ✓
│        │  ├─ P3: 1-2-3-4-17  side c <= 0 ✓
│        │  ├─ P4: 1-2-3-5-8-17  a + b <= c ✓
│        │  ├─ P5: 1-2-3-5-6-8-17  b + c <= a ✓
│        │  ├─ P6: 1-2-3-5-6-7-8-17  a + c <= b ✓
│        │  ├─ P7: 1-2-3-5-6-7-9-10-11-17  a == b == c ✓
│        │  ├─ P8: 1-2-3-5-6-7-9-10-12-17  a == b, b != c ✓
│        │  ├─ P9: 1-2-3-5-6-7-9-13-15-17  b == c ✓
│        │  ├─ P10: 1-2-3-5-6-7-9-13-14-15-17  a == c ✓
│        │  └─ P11: 1-2-3-5-6-7-9-13-14-16-17  all sides different ✓
│        └─ Part 2 - LeaveService independent paths ✓
│           ├─ processRequest()  V(G) = 8 ✓
│           │  ├─ P1: 1-2-10-12-14-16-17  CL approved ✓
│           │  ├─ P2: 1-3-4-5-17  ML requested by contract faculty ✓
│           │  ├─ P3: 1-3-4-6-10-12-14-16-17  ML approved for permanent faculty ✓
│           │  ├─ P4: 1-3-7-8-10-12-14-16-17  OD approved ✓
│           │  ├─ P5: 1-3-7-9-17  unknown leave type ✓
│           │  ├─ P6: 1-2-10-11-17  period has no leave days (only a Sunday) ✓
│           │  ├─ P7: 1-2-10-12-13-17  CL of 4 days exceeds limit of 3 ✓
│           │  ├─ P8: 1-2-10-12-14-15-17  insufficient CL balance ✓
│           │  └─ Sat 31-10 to Mon 02-11: Sunday skipped, 2 days approved ✓
│           └─ countLeaveDays()  V(G) = 3 ✓
│              ├─ Q1: 1-2-6  loop not entered (To date before From date) ✓
│              ├─ Q2: 1-2-3-4-5-2-6  one working day ✓
│              ├─ Q3: 1-2-3-5-2-6  one Sunday (not counted) ✓
│              └─ Loop many times: Mon 02-11 to Sun 08-11 = 6 days ✓
├─ JUnit Jupiter ✓
└─ JUnit Vintage ✓

Test run finished after 360 ms
[         9 containers found      ]
[         0 containers skipped    ]
[         9 containers started    ]
[         0 containers aborted    ]
[         9 containers successful ]
```

```
[         0 containers failed     ]
[        24 tests found           ]
[         0 tests skipped         ]
[        24 tests started         ]
[         0 tests aborted         ]
[        24 tests successful      ]
[         0 tests failed          ]
```

Tests found	Tests successful	Tests failed
24	24	0

3.4 Code Coverage (JaCoCo)

Class	Lines covered	Branches covered	Complexity covered
Triangle	11 / 12	20 / 20	11 / 12
LeaveService	27 / 27	18 / 18	12 / 12
Faculty	12 / 13	no branches	4 / 5

Every branch of Triangle and LeaveService is executed by the suite (100% branch coverage). The only line not executed in Triangle is the implicit default constructor (line 4; classify() is static and no object is created), and in Faculty it is getId(), which the tests do not need.

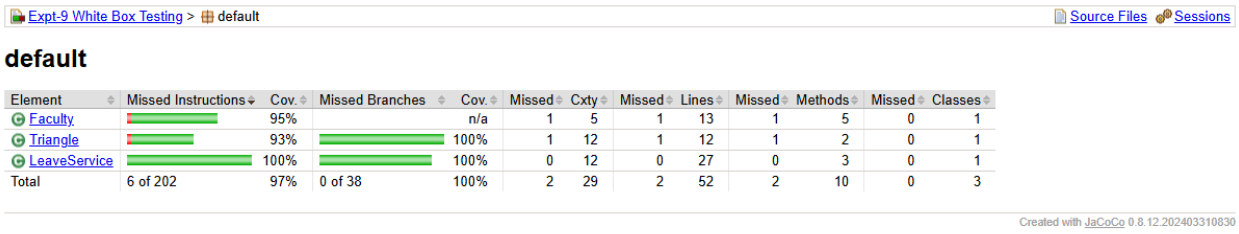


Figure 4: JaCoCo coverage summary for all classes

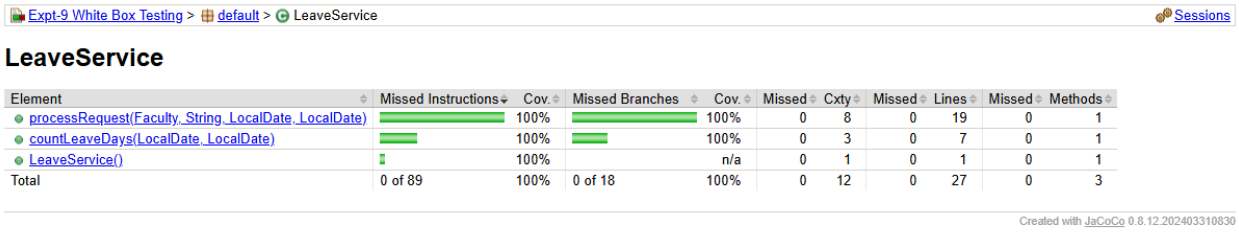


Figure 5: LeaveService - complexity (Cxty) and coverage per method

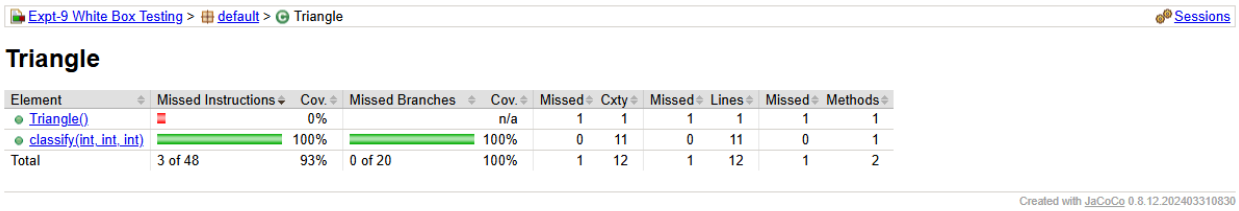


Figure 6: Triangle - complexity (Cxty) and coverage per method

Expt-9 White Box Testing > default > LeaveService.java Sessions

LeaveService.java

```
1.  import java.time.DayOfWeek;
2.  import java.time.LocalDate;
3.
4.  /**
5.   * Part 2: Leave Service of the Online Leave Management System.
6.   * Counts the leave days of a request, checks it against the leave policy,
7.   * approves or rejects it, and updates the faculty member's leave balance.
8.   */
9.  public class LeaveService {
10.
11.     public static final int CL_LIMIT = 3;
12.     public static final int ML_LIMIT = 15;
13.     public static final int OD_LIMIT = 10;
14.
15.     /** Number of leave days from 'from' to 'to' (both inclusive); Sundays are not counted. */
16.     public int countLeaveDays(LocalDate from, LocalDate to) {
17.         int days = 0;
18.         LocalDate day = from;
19.         while (!day.isAfter(to)) {
20.             if (day.getDayOfWeek() != DayOfWeek.SUNDAY) {
21.                 days++;
22.             }
23.             day = day.plusDays(1);
24.         }
25.         return days;
26.     }
27.
28.     /** Applies the leave policy to a request and returns the decision. */
29.     public String processRequest(Faculty faculty, String leaveType, LocalDate from, LocalDate to) {
30.         int limit;
31.         if (leaveType.equals("CL")) {
32.             limit = CL_LIMIT;
33.         } else if (leaveType.equals("ML")) {
34.             if (faculty.isContract()) {
35.                 return "REJECTED: contract faculty are not eligible for ML";
36.             }
37.             limit = ML_LIMIT;
38.         } else if (leaveType.equals("OD")) {
39.             limit = OD_LIMIT;
40.         } else {
41.             return "REJECTED: unknown leave type " + leaveType;
42.         }
43.
44.         int days = countLeaveDays(from, to);
45.         if (days == 0) {
46.             return "REJECTED: no leave days in the selected period";
47.         }
48.         if (days > limit) {
49.             return "REJECTED: " + leaveType + " cannot exceed " + limit + " days per request";
50.         }
51.         if (days > faculty.getBalance(leaveType)) {
52.             return "REJECTED: insufficient " + leaveType + " balance";
53.         }
54.         faculty.deduct(leaveType, days);
55.         int left = faculty.getBalance(leaveType);
56.         return "APPROVED: " + days + " day(s) of " + leaveType + ", balance left " + left;
57.     }
58. }
```

Created with JaCoCo 0.8.12.202403310830

Figure 7: LeaveService.java in the JaCoCo report - green lines and diamonds show executed code and fully covered branches

Expt-9 White Box Testing > default > Triangle.java Sessions

Triangle.java

```
1.  /**
2.   * Part 1: classifies a triangle from the lengths of its three sides.
3.   */
4.  public class Triangle {
5.
6.      public static String classify(int a, int b, int c) {
7.          if (a <= 0 || b <= 0 || c <= 0) {
8.              return "INVALID";
9.          }
10.         if (a + b <= c || b + c <= a || a + c <= b) {
11.             return "NOT A TRIANGLE";
12.         }
13.         if (a == b) {
14.             if (b == c) {
15.                 return "EQUILATERAL";
16.             }
17.             return "ISOSCELES";
18.         }
19.         if (b == c || a == c) {
20.             return "ISOSCELES";
21.         }
22.         return "SCALENE";
23.     }
24. }
```

Created with JaCoCo 0.8.12.202403310830

Figure 8: Triangle.java in the JaCoCo report

Conclusion

Basis path testing was applied to Triangle.classify() (V(G) = 11) and to the LeaveService class of the Online Leave Management System - countLeaveDays() (V(G) = 3) and processRequest() (V(G) = 8). One JUnit test was written for every independent path, plus extra tests for repeated loop execution, and all tests were grouped in WhiteBoxTestSuite. The suite ran 24 tests: 24 passed and 0 failed, and JaCoCo confirmed that every branch of both classes was executed.